Operating Systems

Student Hub | Original study summary | For revision and self-study

Course

CS303 | Student Hub Original Revision Guide

1. Processes and threads

A process owns an address space and operating-system resources. Threads within a process share much of that state but have separate execution contexts. Context switching saves one execution state and restores another.

2. CPU scheduling

Scheduling chooses which ready task runs next. Compare policies using waiting time, turnaround time, response time, fairness, and workload assumptions. A policy that helps one metric can hurt another.

3. Synchronization

A race condition occurs when results depend on timing between concurrent operations. Protect shared invariants with synchronization. A critical section should have mutual exclusion and avoid deadlock and starvation.

4. Deadlocks

Deadlock requires mutual exclusion, hold-and-wait, no preemption, and circular wait. Systems can prevent one condition, avoid unsafe states, detect and recover, or accept the risk depending on the setting.

5. Memory and virtual memory

Virtual addresses are translated to physical memory. Paging divides address spaces into fixed-size pages and frames. A page fault may require loading a page from storage; locality makes caching effective.

6. Files and I/O

File systems map names and directories to stored data and metadata. I/O devices are coordinated through drivers, buffering, and interrupts. Reliability includes permissions, error handling, and recovery after failure.

Study checklist

Draw process states, trace a scheduler, identify a race in a short example, translate a page number and offset, and explain what happens on a page fault.

Operating Systems

Student Hub | Original study summary | For revision and self-study

Practice prompts

1) How do process and thread address spaces differ? 2) What four conditions are needed for deadlock? 3) What is a page fault? 4) Why can adding a lock reduce correctness bugs but also reduce parallelism?

Suggested answers

1) Processes generally have separate address spaces; threads in one process share that address space. 2) Mutual exclusion, hold-and-wait, no preemption, circular wait. 3) A requested virtual page is not currently mapped to resident memory. 4) Locking serializes access to protected shared state.

Practical approach

When debugging concurrency, state the shared invariant, list the operations that access it, then identify the smallest critical section that preserves it.

Further reading

OSTEP and the Linux kernel documentation are linked from the Student Hub Course Materials page.